Projet de classe — Programmation JavaEE

Plateforme de gestion des soutenances de projets de fin d'études (PGS-PFE)

Niveau	Master (M1/M2)
Prérequis	Cours JavaEE, chapitres 1 à 5 (architecture JEE, environnement, Servlet/JSP, MVC, JDBC/JPA)
Modalité	Projet de groupe (3 à 4 étudiants)
Durée indicative	6 semaines
Barème	20 points, détaillés en section 10 ; bonus possible (section 11)

1. Présentation générale du projet

Intitulé du projet : Plateforme de gestion des soutenances de projets de fin d'études (PGS-PFE)

Public : étudiants de Master (M1/M2), dans la continuité du cours « Programmation JavaEE » (chapitres 1 à 5).

Modalité : projet de groupe (3 à 4 étudiants), avec soutenance individuelle des rôles au sein du groupe.

Durée indicative : 6 semaines, en parallèle des séances de cours et de TP.

Ce projet reprend et prolonge, sur un cas plus riche que celui traité en cours (la simple gestion de contacts), l'ensemble des briques technologiques vues dans les 5 chapitres : architecture JEE multi-tiers, Servlet, JSP, JSTL/EL, motif MVC, puis persistance des données avec JDBC et JPA/Hibernate. Le sujet la gestion des soutenances de mémoire a été choisi parce qu'il comporte naturellement plusieurs entités en relation (étudiants, encadreurs, jurys, soutenances), ce qui oblige à sortir du cas d'école à une seule table et à concevoir un vrai modèle de données relationnel.

2. Objectifs pédagogiques

Au terme de ce projet, chaque étudiant doit être capable de :

- Concevoir un modèle de données relationnel à plusieurs entités en relation (1-N et N-N) et le traduire en entités JPA annotées.
- Structurer une application web JEE selon le motif MVC (Servlet contrôleur, JSP vue, classes de modèle et DAO).
- Sécuriser l'accès aux données (PreparedStatement ou JPA) contre l'injection SQL, et gérer une session utilisateur.
- Mettre en place une authentification différenciée selon le rôle de l'utilisateur (administration, encadreur, étudiant).
- Travailler en équipe sur un projet logiciel structuré (répartition des tâches, gestion de version, intégration).
- Présenter et défendre techniquement son travail à l'oral, y compris les choix d'architecture faits.

3. Contexte fonctionnel et acteurs

L'établissement souhaite numériser l'organisation des soutenances de mémoire de fin d'études. Trois types d'utilisateurs interagissent avec la plateforme :

Acteur	Rôle dans l'application
Administrateur (scolarité)	Gère les comptes, les étudiants, les encadreurs, planifie les soutenances et compose les jurys.
Encadreur / membre du jury	Consulte les soutenances où il est convoqué, saisit une note et un avis pour les soutenances qu'il évalue.
Étudiant	Consulte les informations de sa propre soutenance (date, salle, jury) une fois celle-ci planifiée.

Chaque acteur se connecte avec un login et un mot de passe, et n'accède qu'aux fonctionnalités correspondant à son rôle : c'est le premier point sur lequel le projet dépasse le TP de cours, qui ne gérait qu'un seul niveau d'accès.

4. Spécifications fonctionnelles

4.1. Authentification et gestion des rôles

- Page de connexion unique (login + mot de passe) pour les trois types d'acteurs.
- Redirection après connexion vers un tableau de bord différent selon le rôle (administrateur / encadreur / étudiant).
- Aucune page protégée ne doit être accessible sans session valide (vérification systématique côté contrôleur).

4.2. Gestion des étudiants (administrateur)

- Ajouter, modifier, supprimer et lister les étudiants (matricule, nom, prénom, filière, email).
- Rechercher un étudiant par nom ou par matricule.
- Associer à chaque étudiant un sujet de mémoire et un encadreur.

4.3. Gestion des encadreurs et des jurys (administrateur)

- Ajouter, modifier, supprimer et lister les encadreurs (nom, prénom, spécialité, email).
- Composer un jury de soutenance en choisissant plusieurs encadreurs parmi la liste existante (relation plusieurs-à-plusieurs entre soutenances et encadreurs).

4.4. Planification des soutenances (administrateur)

- Créer une soutenance : étudiant concerné, date, heure, salle, jury (au moins deux membres).
- Modifier ou annuler une soutenance déjà planifiée.
- Lister l'ensemble des soutenances, avec un filtre par date ou par étudiant.

4.5. Évaluation (encadreur / membre du jury)

- Un membre du jury connecté ne voit que la liste des soutenances où il est convoqué.
- Il peut saisir, pour chaque soutenance où il siège, une note (sur 20) et un avis textuel.
- Une soutenance est déclarée « évaluée » lorsque tous les membres du jury ont saisi leur note ; la note finale affichée est la moyenne des notes du jury.

4.6. Consultation (étudiant)

- Un étudiant connecté voit uniquement les informations relatives à sa propre soutenance (date, heure, salle, composition du jury).

- Une fois la soutenance évaluée, il peut consulter sa note finale.

5. Modèle de données

Le modèle ci-dessous est un point de départ ; les groupes peuvent l'adapter (ajout d'attributs, de contraintes) tant que les relations essentielles sont respectées.

Entité	Attributs principaux	Relations
Utilisateur	id, login, motDePasse, role (ADMIN / ENCADREUR / ETUDIANT)	Table de base pour l'authentification ; peut être fusionnée avec Étudiant / Encadreur ou reliée à eux, au choix du groupe.
Etudiant	id, matricule, nom, prenom, filiere, email	1 étudiant a 1 sujet et 1 soutenance (0 ou 1) ; 1 étudiant a 1 encadreur.
Encadreur	id, nom, prenom, specialite, email	1 encadreur encadre plusieurs étudiants (1-N) ; peut siéger dans plusieurs jurys (N-N avec Soutenance).
Soutenance	id, dateHeure, salle, statut (PLANIFIEE / EVALUEE / ANNULEE)	1 soutenance concerne 1 étudiant (1-1) ; 1 soutenance a plusieurs membres de jury (N-N avec Encadreur).
Evaluation	id, note, avis	Table de liaison entre Soutenance et Encadreur (jury) : porte la note donnée par chaque membre pour une soutenance donnée.

La relation N-N entre Soutenance et Encadreur (composition du jury) et la table Evaluation associée sont le point technique le plus exigeant du projet : elles n'ont pas d'équivalent dans le TP « Gestion de contacts » du cours et demandent d'utiliser des annotations JPA telles que @ManyToMany ou une entité de liaison explicite, ainsi que des jointures pour les affichages JSP.

6. Contraintes techniques imposées

Ces contraintes reprennent directement les technologies étudiées dans les 5 chapitres du cours :

Couche / aspect	Technologie imposée	Référence dans le cours
Environnement	Eclipse IDE for Enterprise Java and Web, serveur WildFly, base MySQL (via XAMPP ou serveur autonome)	Chapitre 2
Présentation (contrôleur)	Servlets Java (une par cas d'usage ou par famille de cas d'usage), pas de logique métier dans les JSP	Chapitre 3, Chapitre 4
Présentation (vue)	Pages JSP utilisant l'EL (${...}) et, autant que possible, la JSTL (c:forEach, c:if...) plutôt que des scriptlets Java	Chapitre 4
Architecture	Motif MVC strict : Servlet = contrôleur, JSP = vue, classes Java (modèle + DAO/Service) = logique et données	Chapitre 4
Persistance	JDBC avec PreparedStatement pour toute requête paramétrée (aucune concaténation de chaînes dans une requête SQL), ou JPA/Hibernate au choix du groupe pour la couche d'accès aux données	Chapitre 5

Couche / aspect	Technologie imposée	Référence dans le cours
Sécurité	Mots de passe non stockés en clair (au minimum un hachage simple, ex. SHA-256) ; toute requête SQL paramétrée doit passer par PreparedStatement ou l'équivalent JPA	Chapitre 5, section 2.4.2
Session	Gestion de la session utilisateur via HttpSession (setAttribute / getAttribute), avec vérification de la session sur chaque page protégée	Chapitre 4, section 2.4.2

Contrainte explicite de sécurité : un groupe qui construirait une requête SQL par concaténation de chaînes (comme dans l'exemple vulnérable du chapitre 5, section 2.4.2) sur un formulaire de connexion ou de recherche sera pénalisé, même si l'application « fonctionne » — la vulnérabilité par injection SQL correspondante fait partie des points explicitement vérifiés en soutenance.

7. Architecture logicielle attendue

L'application doit être organisée en couches clairement séparées, comme demandé au chapitre 4, section 4.2 : « on ne devrait plus retrouver du code HTML dans une servlet, tout comme ne plus avoir du code java dans une JSP » (au-delà des balises EL/JSTL) :

```
Client (navigateur)
    │ requête HTTP
    ▼
Couche présentation — CONTRÔLEUR (Servlets)
    │ reçoit la requête, appelle les services, prépare les attributs
    ▼
Couche métier — MODÈLE (classes Java : Etudiant, Encadreur, Soutenance...)
    │ logique métier (ex. calcul de la note finale = moyenne du jury)
    ▼
Couche accès aux données — DAO / Repository (JDBC ou JPA)
    │ requêtes SQL paramétrées ou opérations JPA
    ▼
Base de données MySQL

(la réponse redescend : DAO → Modèle → Contrôleur → VUE (JSP) → réponse HTTP au client)
```

Chaque groupe doit produire, pour au moins l'une des entités à relation N-N (Soutenance/Encadreur), une classe DAO (ou repository JPA) dédiée, distincte des servlets, conformément à la recommandation du chapitre 5 : « il faut créer un DAO (Data Access Object), une classe dédiée aux opérations de base de données ».

8. Découpage en jalons

Le projet est découpé en quatre jalons, alignés sur la progression du cours. Chaque jalon donne lieu à une démonstration courte (10-15 minutes) en séance de TP.

Jalon	Semaine	Livrable attendu
J1 — Socle JEE et authentification	Semaine 1-2	Projet Eclipse configuré (WildFly, base MySQL créée), page de connexion fonctionnelle avec Servlet contrôleur, gestion de session et redirection selon le rôle.
J2 — CRUD étudiants et encadreurs	Semaine 3	Ajout / modification / suppression / liste des étudiants et des encadreurs, en JSP + Servlet + DAO JDBC, avec PreparedStatement partout.

Jalon	Semaine	Livrable attendu
J3 — Planification et jury (relation N-N)	Semaine 4-5	Création d'une soutenance avec choix multiple des membres du jury, affichage de la liste des soutenances avec jointures, page de saisie de note par membre du jury connecté.
J4 — Finalisation, migration JPA (au choix) et soutenance	Semaine 6	Calcul et affichage de la note finale, page de consultation étudiant, rapport écrit, et — en bonus — migration d'au moins une entité vers JPA/Hibernate.

9. Livrables et modalités de rendu

- Le code source complet du projet Eclipse (archive .zip ou dépôt Git accessible à l'enseignant).
- Un script SQL de création de la base de données et des tables, avec quelques données de test (au moins 5 étudiants, 3 encadreurs, 3 soutenances).
- Un court rapport écrit (5 à 8 pages) décrivant : le modèle de données choisi, les choix d'architecture, les difficultés rencontrées et comment elles ont été résolues, la répartition du travail au sein du groupe.
- Une soutenance orale de 15 minutes par groupe : démonstration de l'application suivie de questions individuelles sur le code (chaque membre doit pouvoir expliquer la partie qu'il a codée).

10. Grille d'évaluation

Critère	Points	Détail
Fonctionnalités réalisées	6	Authentification par rôle (1,5) ; CRUD étudiants/encadreurs (1,5) ; planification et jury N-N (1,5) ; évaluation et note finale (1,5).
Architecture et qualité du code	5	Respect du MVC, séparation Servlet/JSP/DAO (2) ; absence de code métier dans les JSP (1) ; utilisation cohérente de JSTL/EL (1) ; lisibilité et organisation en packages (1).
Sécurité et robustesse	3	Utilisation systématique de PreparedStatement ou équivalent JPA (2) ; gestion des cas d'erreur (champs vides, session expirée) (1).
Rapport écrit	3	Clarté du modèle de données présenté (1,5) ; qualité de l'analyse des choix et difficultés (1,5).
Soutenance orale	3	Démonstration fonctionnelle (1,5) ; capacité de chaque membre à défendre sa partie du code (1,5).

Total : 20 points. Un bonus de 1 à 2 points peut être accordé pour une extension notable (voir section suivante), sans dépasser 20/20 au total.

11. Pistes d'extension (bonus)

Ces pistes sont facultatives et s'appuient sur les technologies évoquées en fin de cours (chapitres 1 et 4) :

- Migrer l'ensemble de la couche de persistance vers JPA/Hibernate plutôt que JDBC pur (chapitre 5, section 3).
- Remplacer une partie des pages JSP par des composants JSF (chapitre 4, section 5).
- Exposer une API REST (JAX-RS) permettant de consulter la liste des soutenances au format JSON, en vue d'une future application mobile (chapitre 1, section 2.2.3-c).
- Ajouter un tableau de bord statistique pour l'administrateur (nombre de soutenances par mois, moyenne des notes par filière).

- Envoyer un email de confirmation à l'étudiant lors de la planification de sa soutenance (API Java Message Service, mentionnée au chapitre 1).

12. Rappel des correspondances avec le cours

Ce tableau aide chaque groupe à identifier, pour chaque brique du projet, la section du support de cours à relire en priorité.

Brique du projet	**Chapitre et section à consulter**
Architecture générale multi-tiers, rôle du serveur d'application	Chapitre 1, section 2.2
Installation d'Eclipse, WildFly, MySQL	Chapitre 2, section 3
Structure d'une application web JEE (WAR, fichiers publics/privés)	Chapitre 3, section 2
Servlet, doGet/doPost, session, request dispatcher	Chapitre 4, sections 2 et 2.4
JSP, directives, scripts, JSTL, EL	Chapitre 4, section 3
Motif MVC en JEE	Chapitre 4, section 4
JDBC, Statement, PreparedStatement, injection SQL	Chapitre 5, section 2
JPA, annotations, Hibernate, persistence.xml	Chapitre 5, section 3